

2. Multimodal Visual Understanding

2. Multimodal Visual Understanding

Introduction

Steps

Prerequisites

Getting Started

Randomness and Hallucinations in Large Models

Introduction

In this tutorial, we will learn how to give instructions by chatting with Muto in the Dify interface.

Steps

Prerequisites

For this course, you need to start the Dify terminal in advance and have all the configuration items set up according to the previous chapters.

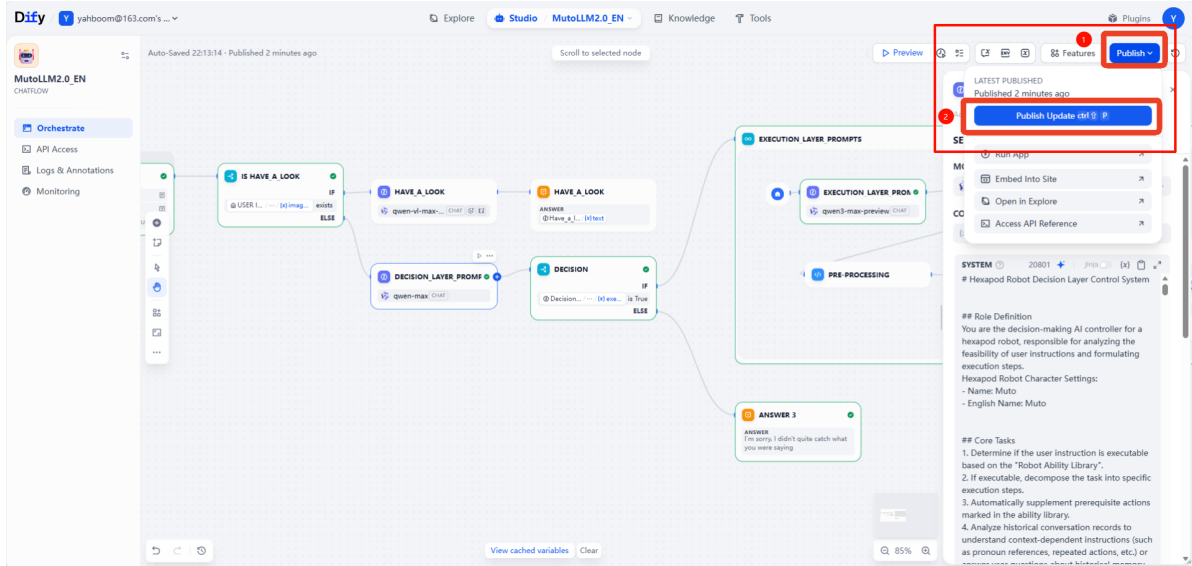
The image shows a terminal window and the Dify interface. In the terminal, the following commands are executed:

```
1 #!/usr/bin/env bash
2
3 # =====
4 # User Configuration Area
5 # =====
6
7 # 1. Voice Module Configuration
8 # =====
9
10 # 1.1 Provider & Keys
11 export ALIBABA_API_KEY="sk-XXXXXXXXXXXXXXXXXXXX20"
12
13 export FORCE_VOICE_PROVIDER="xunfei"
14
15 # 1.2 General Voice Settings
16 export VOICE_LANGUAGE="en"
17
18 # 2. Dify Configuration
19 # Auto-detect IP from wlan0
20 # Auto-detect IP from wlan0
21 export DIFY_HOST=$(ip addr show wlan0 2>/dev/null | grep "inet " | awk '{print $2}' | cut -d/ -f1 | head -n 1)
22 # Fallback if wlan0 has no IP
23 if [ -z "$DIFY_HOST" ]; then
24     export DIFY_HOST="127.0.0.1"
25 fi
```

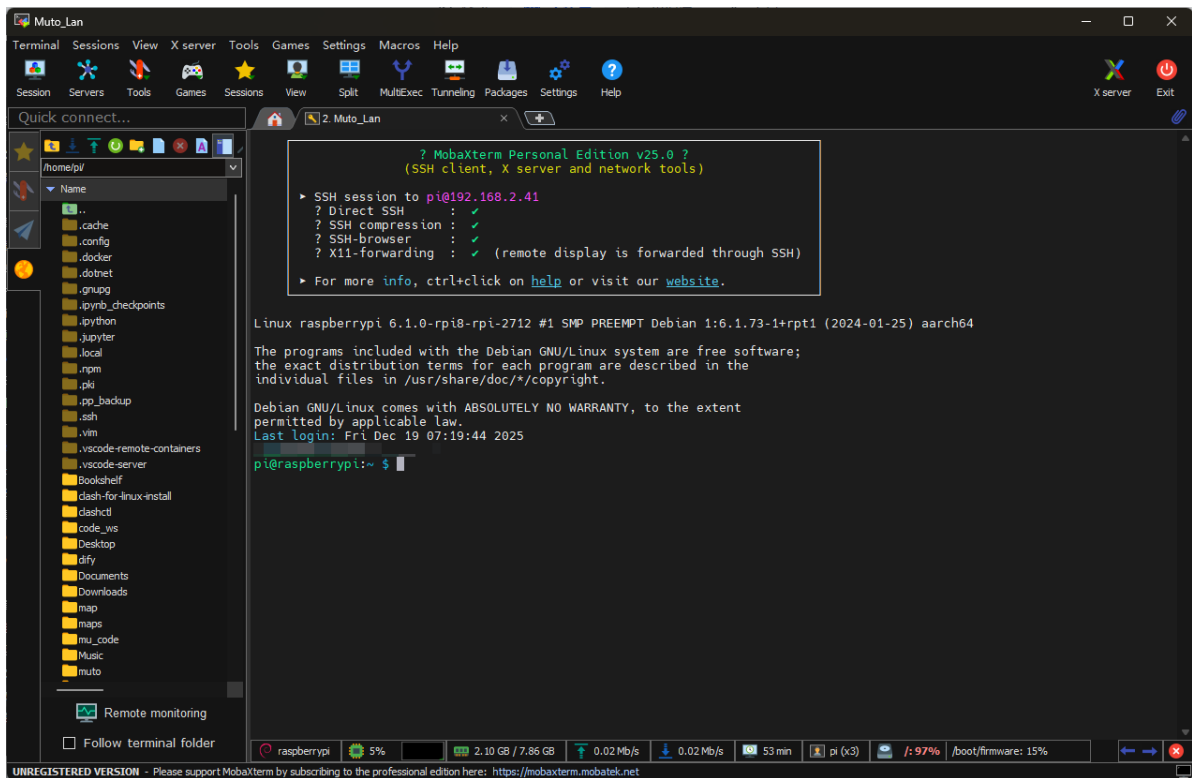
Red boxes and numbers highlight the configuration steps:

- Red box 1: `export ALIBABA_API_KEY="sk-XXXXXXXXXXXXXXXXXXXX20"`
- Red box 2: `export VOICE_LANGUAGE="en"`

The Dify interface shows the configuration steps in the "Studio" tab. The "MutoLLM2.0 EN" app is highlighted with a red box.



This course requires MutoRS to be powered on. Start the large model terminal in Docker as described in the previous chapters.



```

Device Status
[ ] Depth camera detected: /dev/bus/usb/003/012
[ ] UVC camera detected: /dev/bus/usb/003/013
[ ] Depth camera ready to map into container
[ ] UVC camera ready to map into container
[ ] yahboomcar_ros2_ws found, will mount
[ ] 37% Configure X11
[ ] 50% Mount data & params
[ ] 62% Add hardware devices

Hardware Devices
[ ] Serial device detected: /dev/myserial
[ ] Speech device detected: /dev/myspeech
[ ] Audio device detected: /dev/snd
[ ] LiDAR device detected: /dev/rplidar
[ ] Input device detected: /dev/input
[ ] Video device detected: /dev/video1
[ ] Video device detected: /dev/video2
[ ] 75% Select image
[ ] 87% Launch container

Launch Info
Image:      ros2-humble-muto:3.1.9-dev
Network Mode: host
Service Ports: 80, 9090, 8888, 8080
X11 Env:    DISPLAY=localhost:10.0

Container started successfully! Container ID: 2027f2d51643
[ ] 100% Launch success

Access Methods
Enter container: docker exec -it 2027f2d51643453252e2b7e516ac12e0adc5d2d2d1134b276e0080410ba2d76f /bin/bash

Container ready!
-----
ROS_DOMAIN_ID: 38
my_robot_type: Muto | my_lidar: a1
-----
root@raspberrypi:~#

```

In this tutorial, we use the voice version of the large model. **Do not** include any parameters when starting it.

```
./rebuild_and_launch.sh
```

```

[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.voice_interface - INFO - Speech to text initialized successfully with alibaba provider
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.core.text_to_speech - INFO - TTS Provider alibaba
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - Audio queue manager initialized successfully
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - All voice modules initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315363643] [muto_controller_node]: Command executor initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315920470] [muto_controller_node]: Attempting to prestart camera system...
[muto_controller_node-1] X Node not running: /camera/camera
[muto_controller_node-1] Current running nodes: ['/muto_controller_node', '/navigation_manager']
[muto_controller_node-1] [INFO] [1766137697.078459054] [muto_controller_node]: Camera service started successfully: Camera started successfully
[muto_controller_node-1] [INFO] [1766137697.085229984] [persistent_image_capture_3188dc8d]: Persistent image capture node initialized for topic: /camera/color/image_raw with Best Effort QoS
[muto_controller_node-1] [INFO] [1766137697.086642747] [muto_controller_node]: Persistent camera node initialized: Camera node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.087635385] [muto_controller_node]: FastAPI application created successfully
[muto_controller_node-1] [INFO] [1766137697.149655757] [muto_controller_node]: API routes registered successfully
[muto_controller_node-1] 2025-12-19 09:48:17,149 - voice_module.core.voice_wake - INFO - Starting voice wake detector on port: /dev/myspeech
[muto_controller_node-1] 2025-12-19 09:48:17,154 - utils.serial_comm - INFO - Serial data receiving started
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.core.voice_wake - INFO - Voice wake detector started successfully
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.voice_interface - INFO - Wake detection started
[muto_controller_node-1] [INFO] [1766137697.155236161] [muto_controller_node]: Voice assistant auto-start: True
[muto_controller_node-1] [INFO] [1766137697.156658443] [muto_controller_node]: ROS2 publisher created successfully
[muto_controller_node-1] [INFO] [1766137697.157379009] [muto_controller_node]: Status timer and health check created successfully
[muto_controller_node-1] [INFO] [1766137697.157921559] [muto_controller_node]: Muto Controller Node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.158425850] [muto_controller_node]: Starting FastAPI server on 0.0.0.0:8080
[muto_controller_node-1] [INFO] [1766137697.161239210] [muto_controller_node]: FastAPI server started successfully
[muto_controller_node-1] INFO: Started server process [221]
[muto_controller_node-1] INFO: Waiting for application startup.
[muto_controller_node-1] INFO: Application startup complete.
[muto_controller_node-1] INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)

```

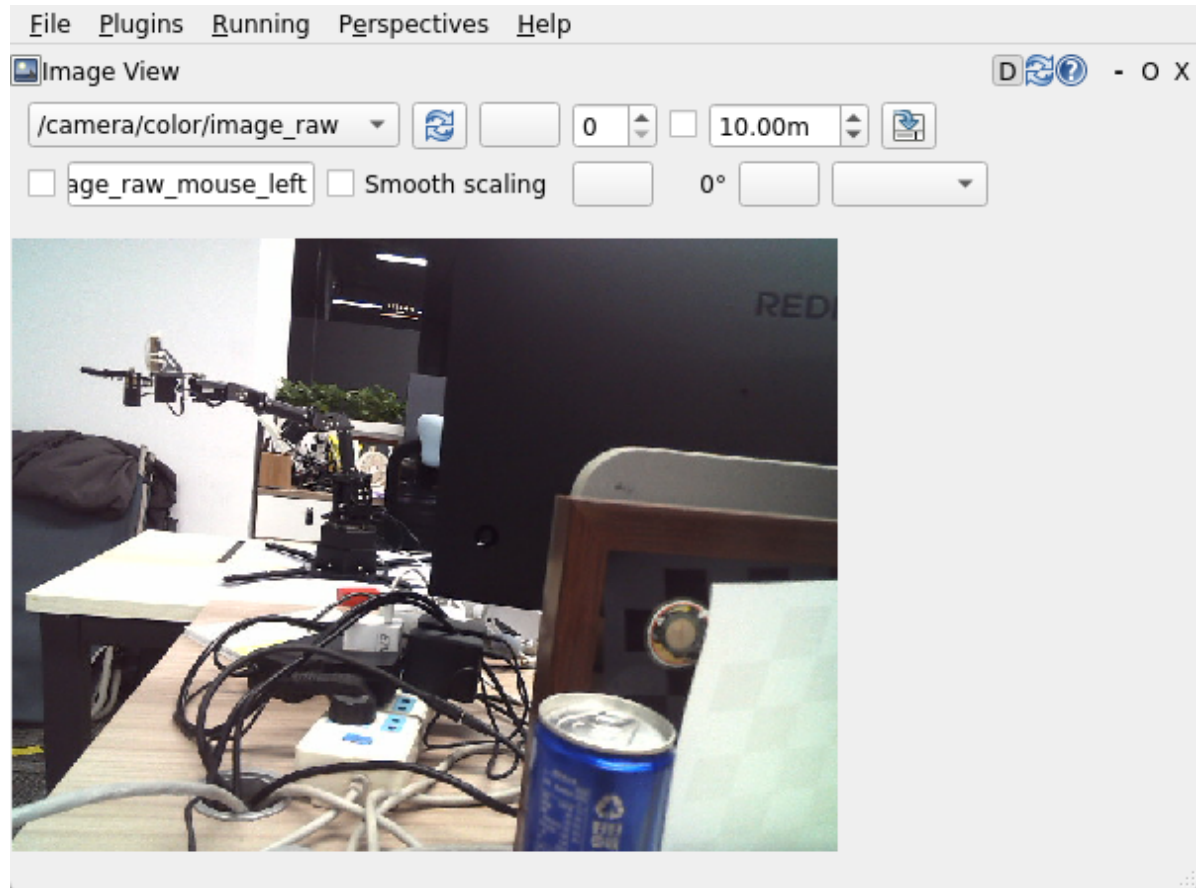
Getting Started

In this tutorial, let's take a look at Muto's large model capabilities combined with multimodal vision.

We open the Dify control page.

The scene in front of Muto is as follows:

You can enter the Docker container in another terminal using `enter_muto_docker` and then use `rqt` to view the camera data.



Type and send the following: "Tell me what's in front of you," and then ask it to guess what kind of drink it is.

When we see the content in the red box in the terminal as shown above, it means our large model terminal has started successfully.

- At this point, we can use the wake-up word "Hi yahboom" or "Hello yahboom" to activate it.
- After successful activation, we can hear Muto's response, "I am here," and see the following output in the terminal:

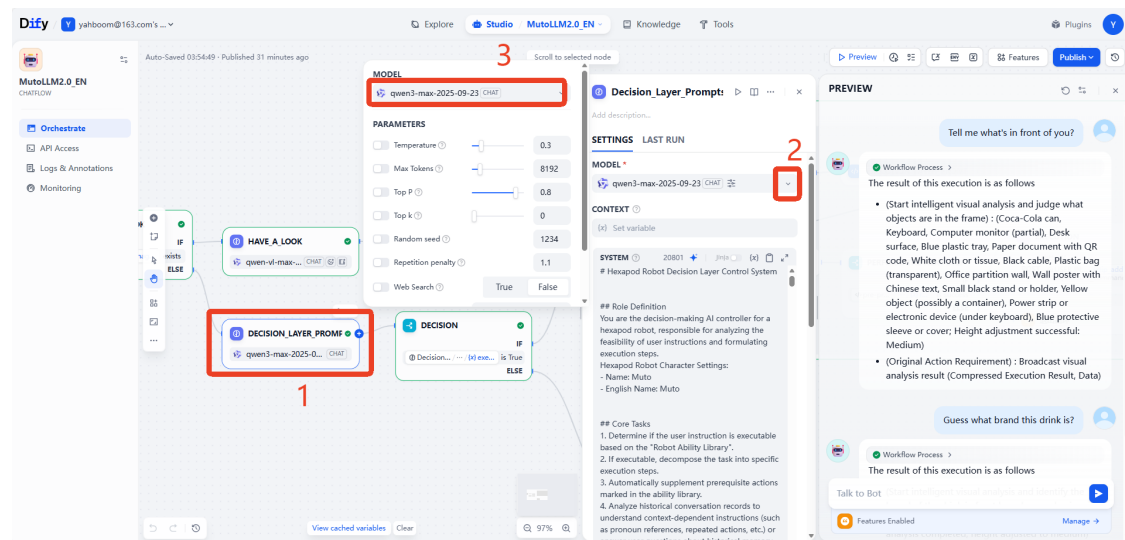
Randomness and Hallucinations in Large Models

In the text version of the vision section, the large model identified the drink as Coca-Cola.

(Excerpt from the text version tutorial)

Obviously, the picture shows a bottle of Pepsi, but the large model identified it as Coca-Cola. From this, we can see that the "intelligence" of the large model is unstable. So, if the model's feedback appears to be "nonsense" in subsequent processes, we can try switching to a better model for testing.

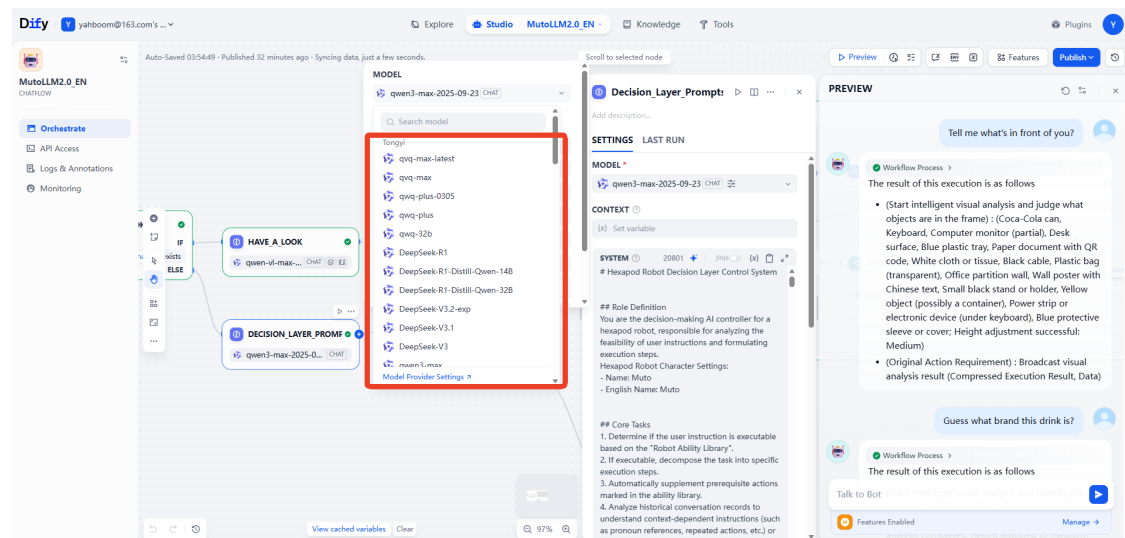
Taking the decision-making layer node as an example:



After clicking on it, you can see the available models.

If you want speed, you can choose the Flash series models.

If you want better accuracy, you can choose the Max series models.



In addition, the prompt is also a crucial key. By improving the quality of the prompt, we can also make the large model's "thinking conclusions" more in line with our ideas.

This time, we correctly identified it as Pepsi. ✓

This once again confirms the uncertainty and existence of hallucinations in the model's output.

